

Vulnerability Research

Calling Conventions

1. What is a stack frame, and how is it used during function calls in C/C++?

A stack is a special region of memory that stores temporary variables which are created by a function.

A stack frame is a data structure that contains;

- one function parameters
- local variables
- return addresses

It helps in maintaining the state of function calls. They will be created when a function is called and then destroyed when a function returns.

```
assignment2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  int multiply(int x, int y) {
5      int result = x * y;
6      return result;
7  }
8
9  int main() {
10     int a = 3;
11     int b = 4;
12     int product = multiply(a,b);
13     cout << "Product: " << product << endl;
14     return 0;
15 }
16 }
```

Each function call, in this case, `main()` and `multiply()`, will get it's own stack frame.

It starts when `main()` function is called and creates the stack frame. This will contain the variables `a,b` and `product`. It will also contain the return address which is where it will go to next once the `main()` process has finished.

The `multiply()` stack frame is created when the `main()` calls `multiply(a,b)`. This stack frame will contain the parameters of x and y, which will get the values from a and b from the main function (x=3, y=4), the local variable result which will store the equation, and then the return address.

The `multiply()` function will calculate the result and return it. The multiply stack frame will then be removed from the stack and the control returned back to the `main()` to get the

output. (Which is 12) This will then print the value of product and the stack frame for *main()* will be popped off the stack and the program exits.

The stack frame structure starts with a *BP - Base Pointer* and ends with *SP - Stack Pointer*.

2. Compare the cdecl and stdcall calling conventions. Who is responsible for cleaning up the stack in each?

For both of these conventions the parameter passing and return values are the same. (Parameters pushed into the stack from right to left) (Return values being stored in the EAX register) but the stack clean ups are different as explained below;

cdecl = Default calling convention for C programs

```
1  #include <iostream>
2  using namespace std;
3
4  int __cdecl add(int x, int y) {
5      return x + y;
6  }
7
8  int main() {
9      int a = 5;
10     int b = 10;
11     int result = add(a,b);
12     cout << "Result: " << result << endl;
13     return 0;
14 }
15
```

The caller is responsible for cleaning up the stack after calling a function. After *add* returns, *main* cleans up the stack. It will adjust the SP to remove the parameters that were pushed. (10 & 5) In assembly after *add* has been returned, *main* will pop the arguments by adjusting *esp* by 8. *esp, 8*.

stdcall = Used in WinAPI functions

```
1  #include <iostream>
2  using namespace std;
3
4  int __stdcall multiply(int x, int y) {
5      return x * y;
6  }
7
8  int main() {
9      int a = 6;
10     int b = 7;
11     int result = multiply(a,b);
12     cout << "Result: " << result << endl;
13     return 0;
14 }
15
```

The callee is responsible for cleaning up the stack (the callee is the function itself) and will clean it up once it finishes executing. The *multiply* function will adjust the SP to remove it's parameters before returning. For example, in assembly a special *ret* instruction with an argument (*ret 8*) will pop the parameters off the stack.

3. Practical Task: Write a simple C program, compile it, and identify the prologue and epilogue in the generated assembly code.

Here's a program that takes two integers and calculates them and returns the answer.

INPUT

```
C a2.c x
C: > Users > cewhi > OneDrive > Desktop > C Programs > C a2.c > main()
1  #include <stdio.h>
2
3  int add(int x, int y) {
4      return x + y;
5  }
6
7
8  int main() {
9      int a = 5, b = 10;
10     int result = add(a,b);
11     printf("Result: %d\n", result);
12     return 0;
13 }
```

OUTPUT

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Result: 15
PS C:\Users\cewhi>
```

Prologues

A prologue will include instructions to the Base Pointer (ebp) and set it to the current Stack Pointer (esp) and allocate the space for local variables.

There are multiple Prologues throughout the code:

_main

```
C: > Users > cewhi > OneDrive > Desktop > C Programs > ASM a2.asm
30  _main PROC C:\Users\cewhi\OneDrive
31  ; File C:\Users\cewhi\OneDrive\Desktop\C Programs\a2.c
32  ; Line 8
33      push ebp
34      mov ebp, esp
35      sub esp, 12 ; 0000000cH
```

_add

```
65  _add PROC
66  ; File C:\Users\cewhi\OneDrive\Desktop\C Programs\a2.
67  ; Line 3
68      push ebp
69      mov ebp, esp
```

_printf

```
84  _printf PROC ; COMDAT
85  ; File C:\Program Files (x86)\Windows Kits\10\include\10.0.22000.0\ucrt\stdio.h
86  ; Line 956
87      push ebp
88      mov ebp, esp
89      sub esp, 8
```

__vfprintf_1

```
123 __vfprintf_1 PROC ; COMDAT
124 ; File C:\Program Files (x86)\Windows Kits\10\include\10.0.22000.0\ucrt\stdio.h
125 ; Line 644
126      push ebp
127      mov ebp, esp
```

__local_stdio_printf_options

```
152 __local_stdio_printf_options PROC ; COMDAT
153 ; File C:\Program Files (x86)\Windows Kits\10\include\10.0.22000.0\ucrt\corecrt_stdio_config.h
154 ; Line 90
155      push ebp
156      mov ebp, esp
```

Epilogues

An epilogue includes instructions to restore the ebp, reset the esp and return the control to the caller.

There is also multiple Epilogues throughout the code:

_main:

```
C: > Users > cewhi > OneDrive > Desktop > C Programs > ASM a2.asm
55 ; Line 13
56     mov esp, ebp
57     pop ebp
58     ret 0
59 _main ENDP
60 _TEXT ENDS
```

_add

```
C: > Users > cewhi > OneDrive > Desktop > C Programs > ASM a2.asm
110 ∨ ; Line 963
111     mov esp, ebp
112     pop ebp
113     ret 0
114 _printf ENDP
```

_printf

```
111     mov esp, ebp
112     pop ebp
113     ret 0
114 _printf ENDP
115 _TEXT ENDS
```

_vfprintf_1

```
144 ; Line 646
145     pop ebp
146     ret 0
147 __vfprintf_1 ENDP
148 _TEXT ENDS
```

__local_stdio_printf_options

```
159 ∨ ; Line 93
160     pop ebp
161     ret 0
162 __local_stdio_printf_options ENDP
163 _TEXT ENDS
164 END
```

Memory Management

1. What is the role of virtual memory, and how does paging help manage it?

Virtual memory is a memory management technique that gives an application the illusion of having a large, continuous block of memory even when RAM is limited. It allows the system to compensate for physical memory shortages. There are two main types of virtual memory, segmentation and paging.

Paging divides the memory into smaller fixed size blocks = pages. When the system runs out of RAM, pages that aren't in use getting moved into the hard drive. The area here is called swap file. This swap file will act as an extension of RAM. When a page is required, it will get swapped back into RAM - This is called Page Swapping. This supports the OS and applications in reallocation of memory, but is a slower process.

2. How do malloc and free differ from new and delete in C/C++?

Malloc/free and new/delete are both used in dynamic memory allocation but do have their differences in functionality.

Malloc/free: Originated in C, they allocate and deallocate raw memory without type safety or initialisation. They do not call constructors or destructors which makes them better for low level memory management. If allocation fails, NULL is returned.

New/delete: Used in C++ are type safe and initialise allocated memory. They do invoke constructors and destructors for objects which makes them ideal for object-orientated programming. If allocation fails, it throws std::bad_alloc allowing for exception handling.

Feature	malloc/free	new/delete
Language	C	C++
Type Safety	Requires explicit casting	Returns pointers of appropriate type
Initialisation	No (raw memory)	Yes (calls constructor)
Destructor Call	No	Yes
Array Syntax	Int *ptr = (int*)malloc(10*sizeof(int));	Int *ptr = new int[10];
Error Handling	Returns NULL	Throws std::bad_alloc

3. Practical Task: Write a C program that demonstrates a memory leak. Use any tool to detect the issue.

Memory Leak Demo

Create the C program and intentionally don't free the allocated memory;

```
C leak.c X
C: > Users > cewhi > OneDrive > Desktop > C Programs > C leak.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void memoryLeak() {
5      int*ptr = (int *)malloc(sizeof(int) * 10) ; //allocate memory
6      // Forget to free the allocated memory
7  }
8
9  int main() {
10     memoryLeak();
11     printf("Memory Leak demo \n");
12     return 0;
13 }
```

Compile the code in the terminal using the flag `-fsanitize=address` to enable the AddressSanitizer;

```
• cfg-vr@codefirstgirls:~/Desktop/Assignments/A2$ gcc -g -fsanitize=address mleak.c -o mleak
```

The output will then display the below. Showing 40bytes leaked in 1 allocation.

```
• cfg-vr@codefirstgirls:~/Desktop/Assignments/A2$ gcc -g -fsanitize=address mleak.c -o mleak
• cfg-vr@codefirstgirls:~/Desktop/Assignments/A2$ ./mleak
Memory Leak Demo

=====
==8175==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 40 byte(s) in 1 object(s) allocated from:
#0 0x77cc95cfbb37 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
#1 0x636c9ed391fe in memoryLeak /home/cfg-vr/Desktop/Assignments/A2/mleak.c:5
#2 0x636c9ed39217 in main /home/cfg-vr/Desktop/Assignments/A2/mleak.c:10
#3 0x77cc9582a1c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#4 0x77cc9582a28a in __libc_start_main_impl ../csu/libc-start.c:360
#5 0x636c9ed39124 in _start (/home/cfg-vr/Desktop/Assignments/A2/mleak+0x1124) (BuildId: 68f0d1aee39ca244ddb8000b072b004e1ddc)

SUMMARY: AddressSanitizer: 40 byte(s) leaked in 1 allocation(s).
• cfg-vr@codefirstgirls:~/Desktop/Assignments/A2$
```

Process Management

1. Explain the difference between a process and a thread.

Computer processes are independent program instances with their own memory space and resources, operating in isolation. Processes are more independent.

Components are:

- Program code (text section)

- Holds the executable instructions of the program
- Fundamental for program execution
- Current activity
 - Program Counter to keep track of the next instruction to execute
 - Processor registers to store the intermediate data and execution state
- Stack
 - Temporary data used during program execution
 - Contains function parameters, return addresses and local variables
- Data section
 - Global variables - accessible throughout the programs lifecycle
- Heap
 - Dynamic memory allocation which is used for variables whose size and duration are determined at runtime

Threads are smaller units within processes that share the same memory making communication easier but requiring careful synchronisation. Threads enable concurrent execution and resource sharing within a process.

Each thread will have these components:

- Thread ID
 - To help identify, these are unique
- Program Counter
 - Points to the next instruction to be executed
- Registers
 - CPU registers that store the current state of execution
- Stack
 - They will all have their own stack for function calls and local variables

	Process	Thread
Resource Allocation	Has it's own resources, seperate memory space, file handles and environment. When the process starts, the OS allocates these resources to it	Threads within the same process share resources. This sharing allows quicker access but requires careful handling to avoid conflicts
Communication	Complex, often requires mechanisms like IPC, sockets, or shared files, as each process is independent.	As threads within a process share the same memory space, they communicate directly with each other.
Overhead	Heavier, consuming more memory and resources because of their isolated environment. Context switching is also slower due to this isolation	Lighter because of the shared memory and resources. This means creating or switching between threads is quicker.
Fault Isolation	If a process crashes it will not affect other processes as they are isolated	When a thread encounters an issue within a process, it could potentially effect the entire process since they share the same memory and resources
Uses	Ideal when running completely independent tasks, especially those that require substantial isolation or where failure containment is important	Commonly used when tasks are closely related and need to share data frequently, such as handling multiple tasks within a single application

2. What are the main risks of improper process management, such as zombie processes?

When a process isn't correctly managed it will cause a number of issues. From resource exhaustion to operational overhead problems. The zombie process is when a process has finished executing but remains in the process table taking up a slot even though it no longer actively runs or consumes CPU resources. This is happening because the processe's parent did not recieve the correct exit status.

Some of the risks are:

- **Memory Leak** - Processes consume resources in the systems process table. There is a limited number of entries here and if there's an excess of zombie processes running they are taking up a place that new processes may require to run
- **CPU Wastage** - When processes are improperly managed, especially orphaned or stuck processes there will be unnecessarily CPU usage which could slow down other applications and reduce the systems efficiency.
- **Process Table Overflow** - When there are too many zombie processes occupying the tables this can make the system unresponsive and users may need to reboot the machine to regain control.
- **System Crashes** - For critical systems an overflowed process table could lead to a crash which will interrupt essential services or applications.
- **Exposure to Exploits** - When processes are improperly terminated or orphaned it can expose the system to security risks as attackers could exploit these to inject malicious code or execute privilege escalation attacks.
- **Data Loss or Corruption to the Data** - Mismanaged processes could interfere with I/O operations leading to partial or incorrect read/write operations which could lead to data loss or corrupted data.
- **Slow System Response** - When too many processes are running or consuming memory unnecessarily, response time will be affected
- **Application Latency** - If improper process management continues it will consume too many system resources and performance of other applications could be delayed, cause freezes or even degrade.
- **Increased Maintenance Costs** - Having to manually identify and kill zombie processes can be time consuming, especially if the system is large and complex where the issue is frequently happening.
- **Reduced System Uptime** - Constant monitoring and rebooting due to process management issues will reduce uptime and disrupt service ability.

3. Practical Task: Write a simple C/C++ program and show process information when you run that program using Process Explorer or ps/top/htop command depending on where you are running it

The following program will increment a counter on an infinite loop, printing the counter every 1,000,000 iterations to show it's actively running. It will sleep for 1 second after every print to make CPU usage more visible.

Input

C: > Users > cewhi > OneDrive > Desktop > C Programs >  process.cpp >  main()

```
1  #include <iostream>
2  #ifdef _WIN32
3  #include <windows.h> // Windows sleep function
4  #else
5  #include <unistd.h> // Unix sleep function
6  #endif
7
8  using namespace std;
9
10 int main() {
11     int counter = 0;
12     while (true) {
13         counter++;
14         if (counter % 1000000 == 0) {
15             cout << "Counter: " << counter << endl;
16
17             #ifdef _WIN32
18                 Sleep(1000);
19             #else
20                 Sleep(1);
21             #endif
22         }
23     }
24     return 0;
25 }
```

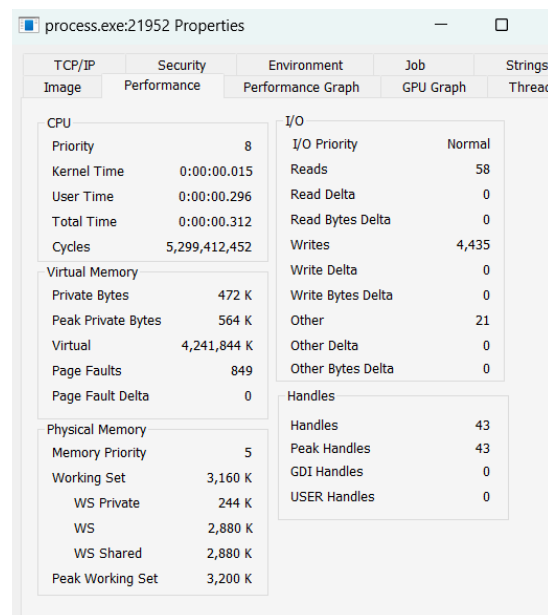
Output

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
Counter: 1000000
Counter: 2000000
Counter: 3000000
Counter: 4000000
Counter: 5000000
Counter: 6000000
Counter: 7000000
Counter: 8000000
```

This will just keep on going and going and going.....

I've used Process Explorer to find the properties of process.exe. This is helpful for analysing how much system resources is being used when running processes.



Process	CPU	Private Bytes	Working Set	PID	Description	Company Name
HPEnhancedLighting.Bg.exe	< 0.01	28,352 K	23,336 K	19784	HPEnhancedLighting.Bg	HP Inc.
Code.exe		66,104 K	69,536 K	13016	Visual Studio Code	Microsoft Corporation
conhost.exe	< 0.01	1,324 K	6,968 K	12100	Console Window Host	Microsoft Corporation
powershell.exe	< 0.01	54,272 K	52,764 K	12364	Windows PowerShell	Microsoft Corporation
WindowsTerminal.exe	0.01	97,272 K	142,896 K	12876		
OpenConsole.exe	< 0.01	2,324 K	10,204 K	8408		
cmd.exe	< 0.01	4,168 K	6,856 K	15096	Windows Command Process...	Microsoft Corporation
process.exe	< 0.01	472 K	3,160 K	21952		
process.exe	< 0.01	460 K	3,132 K	21080		

process.exe

< 0.01

508 K

3,180 K

21952

Handles

DLLs

Threads

Stack

Module

Terminate

State	Wait Reason	TID	User Time	Kernel Time	CPU	CPU Time	Start Time	Start Address	Base Pri	Dyn Pri	Service	Context Switc...	Suspe...	Ideal C...
Waiting	DelayExecution	2968	00:00:00	00:00:00	0.01	00:00:00	11/01/24 21:05:01	process.exe!LT+13560(m...	8	8		5,321		1
Waiting	WrQueue	10844	00:00:00	00:00:00		00:00:00	11/01/24 21:10:01	ntdll.dll!RtlClearThreadW...	8	8		2		2

References:

- [Virtual Memory in Operating System - GeeksforGeeks](#)
- https://summermasters2024.slack.com/files/U07L2UT5CU9/F07T7DT4AKT/2.2_memorymanagement.pptx.pdf
- [Threads vs. Processes: How They Work Within Your Program](#)
- [CFG Vulnerability Research - 2024 - Zoom](#) - Process Management
- [How to Use Process Monitor and Process Explorer](#)
- [Configure launch.json for C/C++ debugging in Visual Studio Code](#)
- [Setup VSCode to run and debug C / C++ code](#)
- [c - How do I use valgrind to find memory leaks? - Stack Overflow](#)